

第 03 讲：Coder 基座模型路线

Coding Agent Notes (June 2026)

1 核心图景

Coder 基座模型的发展路线，不是从“会写代码”平滑升级到“会做软件工程”。中间至少隔着五个能力台阶：补全、编辑、长上下文、仓库结构、指令对齐。每一个台阶都对应一种真实工作需求，也对应一种新的训练数据或训练目标。



Agent-ready coder model = code-heavy pretraining + edit-aware objective + repo context + instruction alignment + deployment cost discipline.

这个公式的重点不是把所有模型都归到一个排行榜，而是说明能力从哪里来。普通 code completion 主要回答“下一段代码是什么”；coding agent 需要回答“在这个仓库、这个 bug、这个测试失败、这个工具反馈之后，下一步应该怎么改”。这两件事共享语言建模能力，但训练压力完全不同。

因此，读 coder 模型报告时，不应该只问参数量和 benchmark 分数，还要问：模型是否学过在已有代码中间补洞？是否能读到足够长的上下文？训练数据是否保留仓库结构？后训练是否教它听懂人类任务，而不是只延续代码文本？推理成本是否能放进真实工具循环？

Insight. 代码模型的进步不是单一维度的 scale-up。真正重要的是：训练目标逐步贴近软件工作的接口。Completion 对应“写下一个 token”；FIM 对应“在已有文件中编辑”；repo-level data 对应“跨文件理解”；instruction tuning 对应“把代码能力接到人类任务”。

1.1 它在 agent stack 里的位置

在长期 coding agent 里，基座模型位于最底层。它提供代码理解、代码生成、错误解释、计划草拟和自然语言沟通能力。但它不是完整 agent。模型不会自动知道该读哪个文件、怎么运行测试、何时停止、如何处理权限，也不会自动把 patch 交付成 PR。这些属于 harness、上下文、工具和验证层。这一区分很重要：强 coder 模型可以降低 harness 负担，但不会消除 harness。相反，越强的基座模型越需要清楚的行动接口，否则它的能力会浪费在错误上下文、脆弱命令、冗长日志和不稳定验证里。

2 能力阶梯

第一层是 completion。

最早的代码模型主要学习从左到右预测代码。它可以根据函数签名、注释或前文补出一段合理实现。这个能力对单文件补全、简单函数生成和 boilerplate 很有用，但它天然偏向“追加文本”。真实软件工程更多是在已有代码中间修改：插入一个分支、改一个条件、替换一个 API、移动一段逻辑。只会从左到右补全的模型，在编辑任务上会被接口限制住。

第二层是 infilling。

Fill-In-the-Middle (FIM) 把训练样本拆成 prefix、middle、suffix，让模型在看到前后文的情况下预测中间缺口。IDE 补全、局部 patch、函数内部插入、类型修复，都更接近 FIM 而不是纯 completion。Code Llama、DeepSeek-Coder、StarCoder2、Qwen2.5-Coder 都把 FIM 当成关键训练目标或格式。

第三层是 long context。

仓库级任务需要模型看到更多代码、错误日志、文档和测试。长上下文让模型有机会把多个文件、长函数、调用链和报错信息放在同一次推理里。但“能塞进去”不等于“能用好”。如果上下文没有结构，长 prompt 只会变成更长的噪声。

第四层是 repo-aware。

Repo-aware 不是简单把很多文件拼起来，而是让模型看到文件边界、路径、仓库名、依赖顺序、PR 或 issue 结构。DeepSeek-Coder 用项目级依赖排序和 repo-level dedup；StarCoder2 和 Qwen2.5-Coder 都显式引入 repository-context 或 repo-level format。这个层次开始接近 coding agent 的真实输入。

第五层是 alignment。

代码基座模型学到的是文本分布；coding assistant 需要听懂人类要求，解释取舍，遵守格式，避免危险输出，并在多轮对话里保持目标。Instruction tuning、SFT、DPO、reward model 或 RL 都在做这件事。它们把“会写代码”转成“能作为助手回应任务”。

2.1 一个最小任务如何穿过这些台阶

设想一个真实但很小的 bug：用户说“登录表单在空密码时没有显示错误”。如果这是单轮代码题，模型只需要写一个函数；但在真实仓库里，它要先定位表单组件，读验证逻辑，找到测试或故事文件，再判断错误是 UI 状态、schema validation 还是提交逻辑。

completion 能力让模型补出一个看似合理的条件分支，例如 `if password == ""`。但如果这个分支要插在已有 reducer 或 form schema 中间，模型需要 infilling。它必须同时看前面的状态定义和后面的 submit handler，不能只看光标前文。

如果验证逻辑分散在多个文件里，long context 才有意义。模型可能需要同时看 `LoginForm.tsx`、`validation.ts`、`auth.test.ts` 和错误信息。但仅有长 context 仍不够：如果这些文件没有路径、边界和依赖关系，模型很容易把测试文件里的 helper 当成生产逻辑，或者在错误文件中 patch。

repo-aware 数据让模型更熟悉这种结构：路径代表模块，文件边界代表职责，测试和实现之间有调用关系。instruction alignment 则让模型理解用户真正要的是“修复行为并解释修改”，而不是随意生成一段新代码。

这个例子说明，agent-ready 不是一个神秘属性，而是多个能力在同一任务中同时被调用。缺一层，任务就会退化：没有 FIM，模型难以编辑；没有 repo context，模型难以定位；没有 instruction alignment，模型难以把代码能力接到人类意图；没有 harness，模型仍然不能验证 patch。

3 Code Llama：从通用模型到代码专用模型

Code Llama 是一个清晰的起点：它不是从零训练纯代码模型，而是从 Llama 2 继续做 code-heavy training。这个选择说明一个基本判断：代码模型仍然需要通用语言能力。用户 issue、README、错误日志、文档、StackOverflow 式解释都不是纯代码；如果模型只会代码语法，不会自然语言和任务语义，它无法成为好的 coding assistant。

Code Llama 的路线可以拆成三个分支：基础 Code Llama、Python-specialized、Instruct。基础模型面向通用代码生成；Python 分支额外用 Python-heavy 数据专门化；Instruct 分支把模型调到更适合人类命令和安全约束。这个分叉很有启发：同一个 base capability 可以被不同 downstream interface 重

新塑形。

分支	训练重点	解决的问题
Code Llama	code-heavy continued training	让通用 LLM 更懂代码
Python	Python-heavy specialization	提高单语言深度
Instruct	instruction/safety tuning	接人类任务和对话

Code Llama 还把两个后来反复出现的设计推到台前：FIM 和 long-context fine-tuning。7B、13B 和 70B general models 支持 infilling，长上下文阶段把训练序列提高到 16K，并报告能改善更大输入上的表现。对 coding agent 来说，这两个能力不是“加分项”，而是进入真实编辑环境的前提。

4 StarCoder2：开放数据和 repo-context

StarCoder2 的重要性不只在模型尺寸。它把 code model 的讨论拉回到数据治理和数据结构上。The Stack v2 来自 Software Heritage 的源代码档案，覆盖 619 种编程语言，并加入 PR、issues、notebooks、代码文档、数学和推理数据。报告还强调去重、PII redaction、malicious code filtering、opt-out 和 SWHIDs。这些听起来像数据工程细节，但它们直接影响模型学到什么，也影响研究者能否审计模型。

StarCoder2 训练 3B、7B、15B 模型，采用 4K base training 再到 16K long-context stage 的两阶段路线。更关键的是，它从 file-context 转向 repository-context：不是把随机文件拼进上下文，而是把同一仓库的文件组织成一个样本，并在 repo-context 里应用 file-level FIM。



这张图表达的是：代码模型的数据不只是“更多代码文件”。PR 和 issues 暴露人类协作中的修改意图；notebooks 和文档提供 code-text grounding；math 和自然语言保留推理与解释能力；synthetic data 提供可控任务分布。模型最终学到的，是这些数据形态共同定义的开发语境。

Insight. 训练数据的单位会改变模型的世界观。file-level 数据让模型把代码看成独立文本；repo-level 数据让模型看到路径、边界和跨文件关系；PR/issue 数据让模型看到“为什么修改”。这些信号比单纯增加 token 数更接近软件工程。

4.1 为什么不是 100% Code

直觉上，训练 code model 应该尽量提高代码比例。但几篇报告都在不同方式上反驳了这个直觉。Code Llama 在 code-heavy 数据之外保留 code-related natural language；DeepSeek-Coder 使用 10% code-related English 和少量中文；DeepSeek-Coder-V2 使用 code、math、natural language 混合；Qwen2.5-Coder 最终选择 70% Code、20% Text、10% Math。

原因是软件工程任务不是纯代码分布。issue 是自然语言，测试失败是自然语言和符号混合，README 和文档解释 API，PR 讨论解释为什么要改，数学数据帮助模型保留形式推理能力。一个只看代码文件的模型，可能语法很强，却不擅长把用户意图、错误描述和代码结构连接起来。

数据形态	提供的信号	缺失时的风险
Source code	语法、API、实现惯例	不会写可靠代码
Docs / Q&A	解释、用法、错误描述	不懂任务语义
PR / issues	修改动机和审查语言	只会写，不会修
Math data	形式推理和约束推导	复杂逻辑易错
Synthetic data	可控覆盖和稀缺任务	长尾任务不足

因此，code model 的数据混合不是“代码 vs 非代码”的二选一，而是在训练一个能参与软件工程语言游戏的模型。真实开发者也不是只读源文件：他们读 issue、文档、commit message、测试名、错误日志和 reviewer comment。模型要接近这种工作方式，训练数据也必须包含这些信号。

5 DeepSeek-Coder：项目级代码语料

DeepSeek-Coder 把 repo-aware 的思路推进得更直接。它从零训练 1.3B 到 33B 的 base 和 instruct 模型，训练语料包含 87 种编程语言、2T tokens。数据配比是 87% source code、10% code-related English natural language、3% Chinese natural language。这个配比本身就说明：代码能力不是只从代码里来，文档、问答和自然语言解释也是代码能力的一部分。

最值得关注的是项目级数据构造。DeepSeek-Coder 不只是抓取文件，而是尝试解析同一仓库中文件之间的依赖关系，把依赖更早的文件排在使用它们的文件之前，并保留 file path 信息。这样模型看到的上下文更像真实项目：一个文件里的符号来自另一个文件，路径本身也携带模块结构。

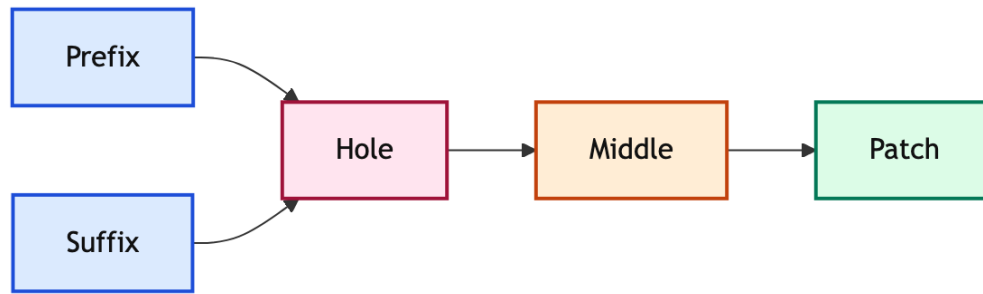
另一个细节是 repo-level dedup。很多代码语料会做 file-level 去重，但如果把仓库里的某些文件删掉，可能破坏项目结构。DeepSeek-Coder 选择把仓库级拼接样本作为去重单位，以尽量保留仓库结构。这对 coding agent 很关键：agent 处理 bug 时，正确线索常常分散在多个文件里。如果训练时把这些关系打散，模型就很难学到跨文件定位的习惯。

数据处理	直观目的	对模型的影响
依赖排序	让被调用文件更早出现	更接近跨文件阅读
路径保留	给出模块和文件位置	提高定位意识
repo 去重	保留仓库整体结构	避免打碎项目样本
质量筛选	过滤语法差和低质量代码	降低噪声学习

DeepSeek-Coder 还给 FIM 做了很有用的 ablation：100% FIM 对 infilling 最强，但会损害普通 code completion；最终选择 50% PSM 作为折中。这说明训练目标不是免费叠加。每一种能力都会占用模型容量和训练分布，过度优化一种接口可能削弱另一种接口。

6 FIM：为什么编辑不是补全

FIM 的形式很简单：给模型 prefix 和 suffix，让它预测 middle。真正重要的是它改变了模型面对代码的方式。Completion 假设光标在末尾；FIM 假设光标在中间。软件工程里的大多数改动恰好发生在中间。



一个 bug fix 往往不是新写一个函数，而是在旧函数里改一行判断；不是追加一个新文件，而是在已有文件中插入一个 import；不是从头写测试，而是补上缺失 case。FIM 让模型在训练时反复看到“前文和后文同时约束中间缺口”的任务。它更接近 patch generation 和 IDE editing。

但 FIM 也有 tradeoff。DeepSeek-Coder 的实验显示，完全偏向 FIM 会削弱普通 completion。原因并不神秘：模型训练分布改变后，模型会更擅长重排后的 prefix-suffix-middle 格式，却可能不如以前那样自然地左到右续写。对产品而言，这意味着训练配比必须服务于目标工作流：IDE 补全、patch 编辑、agent 自主修复，对 FIM 的需求并不相同。

Completion asks: given code before the cursor, what comes next?

FIM asks: given code before and after the cursor, what belongs in the gap?

7 Long Context: 仓库不是长 prompt

Long context 解决的是容量问题：模型一次能看到多少 token。Repo context 解决的是结构问题：这些 token 是否以有意义的方式组织。两者相关，但不能混为一谈。

Code Llama 通过 long-context fine-tuning 让模型适应 16K 训练序列，并报告在更长输入上有改进。StarCoder2 先 4K base training，再用 16K 做 long-context stage。DeepSeek-Coder 把可靠上下文推进到 16K。DeepSeek-Coder-V2 和 Qwen2.5-Coder 又把 128K 作为重要能力目标。

但是，如果把整个仓库粗暴塞进 128K，模型仍然可能失败。它可能读到大量无关文件，忽略真正相关函数；也可能看到路径但不知道依赖顺序；还可能被日志、文档和重复代码稀释注意力。长上下文只是让模型有机会看见更多信息，不保证它选择正确信息。

file boundary 是结构信号。

Qwen2.5-Coder 引入 repo_name 和 file_sep 这类特殊 token，目的就是告诉模型：下面不是连续文本，而是仓库里的多个文件。StarCoder2 的 repo-context FIM 也使用 repository metadata 和 file separator。DeepSeek-Coder 用路径注释保留 file path。这些设计都在给长上下文加结构。

dependency order 是因果信号。

如果文件 A 定义 API，文件 B 调用 API，让 A 更早出现会帮助模型建立“定义先于使用”的阅读顺序。真实开发者也会这么做：先看定义，再看调用，再看测试。repo-level 数据把这种阅读路径写进训练样本。

Insight. 长上下文不是把工作区变成一大段 prompt，而是把工作区压缩成可读结构。没有路径、边界、依赖、检索和验证，128K context 也可能只是更贵的噪声容器。

8 DeepSeek-Coder-V2: MoE 与 128K

DeepSeek-Coder-V2 展示了另一条路线：用 MoE 增大总容量，同时控制单次推理的 active parameter。报告给出两个规模：16B total / 2.4B active，以及 236B total / 21B active。它从 DeepSeek-V2 的中间 checkpoint 继续预训练，额外使用 6T tokens；加上原 checkpoint 的 4.2T，总暴露 token 达到 10.2T。

MoE 对 coding agent 很有现实意义。长期任务通常要多轮调用模型：搜索、读取、编辑、测试、解释失败、再编辑。如果每一步都调用一个高延迟高成本 dense 模型，agent 的总成本会迅速放大。MoE 试图把总容量和单次计算成本拆开，使模型拥有更大的知识与能力池，但每次只激活一部分专家。

模型	总参数	激活参数	上下文目标
DeepSeek-Coder-V2-Lite	16B	2.4B	128K
DeepSeek-Coder-V2	236B	21B	128K

DeepSeek-Coder-V2 同时把语言覆盖从 86 种扩到 338 种，并把 context 从 16K 扩到 128K。它还保留数学和自然语言数据，以避免模型只会代码而丢失推理与通用理解。对 agent 来说，这些能力并不是互相独立的：issue 是自然语言，测试失败需要推理，代码修复需要语言和结构同时工作。

9 Qwen2.5-Coder: file 到 repo 到 alignment

Qwen2.5-Coder 把这条路线整理得很清楚：先 file-level pretraining，再 repo-level pretraining，最后 instruction tuning。它有 0.5B、1.5B、3B、7B、14B、32B 六个规模，所有规模都报告 5.5T trained tokens。数据包括 source code、text-code grounding、synthetic data、math data 和 general text。



file-level pretraining 使用 8,192-token 序列，目标是 next-token prediction 和 FIM。这个阶段主要让模型学会单文件内部的语法、API、局部逻辑和编辑模式。repo-level pretraining 把 context 从 8K 扩到 32K，并通过 YARN 让模型能够处理到 128K；它使用约 300B long-context code data，并把 file-level FIM 扩展成 repo-level FIM。最后，SFT 和 DPO 把 base model 转成更适合人类任务的 instruct model。

Qwen2.5-Coder 的数据配比也值得看：最终选择 70% Code、20% Text、10% Math。直觉上，code model 应该尽量多喂代码；但报告中的实验显示，合适比例的 text 和 math 能帮助平均能力。这说明 coder model 不只是代码语法模型，它还需要任务解释、符号推理、错误分析和文档理解。

阶段	训练材料	获得的能力
File pretrain	5.2T file-level data	局部代码生成和编辑
Repo pretrain	300B long-context code	跨文件结构和仓库上下文
SFT / DPO	instruction preference data	人类任务和助手行为

这条路线也解释了为什么 Qwen3-Coder-Next 这样的系统会继续往可执行任务训练走。Qwen2.5-Coder 已经把模型推到 repo-aware 和 instruction-aware；但 coding agent 还需要从“会回答任务”进一步走向“能在环境中执行任务、接收验证反馈、从失败中迭代”。那属于 agentic training，而不是单纯 foundation model training。

10 Base、Instruct 与 Agent-Ready

很多模型报告会同时发布 base、instruct 或 chat 版本。它们不是简单的“同一个模型加 prompt”，而是面向不同接口的模型状态。

Base model 最接近预训练分布，适合补全、继续写代码、作为后训练起点。它通常保留更强的原始 completion 行为，但不会天然遵守用户格式，也不会稳定解释自己的选择。Instruct model 经过 SFT、DPO 或类似对齐训练，更擅长回答自然语言任务、给出解释、遵守格式和处理多轮对话。Chat model 往往进一步强调对话安全、拒答边界和人类偏好。

Agent-ready base 不是一个标准论文标签，而是系统设计上的判断：这个模型是否适合被放进工具循环里，反复接收 observation、生成 action、解释失败并继续推进。它可能是 instruct model，也可能是专门为 agent harness 后训练过的模型。关键不是名字，而是它能否稳定承担工具循环中的角色。

版本	适合接口	常见风险
Base	补全、继续预训练	不稳定遵守任务
Instruct	人类命令、解释、格式	可能改变补全习惯
Chat	多轮对话和安全边界	可能过度解释或拒答
Agent-ready	工具循环和验证反馈	仍依赖 harness

这个区分能避免一个常见误读：用 instruct 模型的对话能力来替代代码能力，或者用 base 模型的补全能力来替代任务执行能力。对 coding agent 来说，最好的模型不是抽象意义上的“最会写代码”，而是最适合当前 action space、observation format、验证策略和预算的模型。

11 对 Coding Agent 的启发

强模型让 harness 更简单，但不会让 harness 消失。

如果基座模型已经学过 FIM、long context、repo-level structure 和 instruction following，harness 可以少做一些脆弱的 prompt workaround。但 harness 仍然必须负责上下文选择、工具调用、权限控制、测试运行、轨迹记录和结果验证。模型越强，harness 越应该把它的能力暴露成稳定动作，而不是用混乱上下文消耗它。

模型路线决定 agent 能稳定委托什么。

一个只做 completion 的模型，适合 autocomplete，不适合自主 patch。一个 FIM 强但长上下文弱的模型，适合局部编辑，不适合跨文件修复。一个 repo-aware 但 instruction 弱的模型，可能看得懂文件关系，却不稳定遵守用户要求。读模型报告时，要把训练路线翻译成 agent 可委托能力。

部署成本也是模型能力的一部分。

Coding agent 的模型调用不是一次性的。一个长任务可能需要几十轮工具交互，每轮都要读观察、计划下一步、生成命令或 patch。MoE、小模型、多尺寸系列和 active parameter 设计都在回答同一个问题：模型是否能以可接受延迟和成本进入真实工作循环。

Insight. “最强 coder model”不是唯一答案。产品里的 agent 可能需要低延迟本地模型、长上下文云模型、强推理模型或便宜的候选生成模型。基座模型路线的价值在于帮助我们判断：某个模型适合放在 agent 系统的哪一层。

12 怎样读 Coder 模型报告

读 coder 模型报告时，可以把所有细节归到五个问题。第一，模型是从通用 LLM 继续训练，还是从零训练代码模型？前者通常保留通用语言能力，后者可以更彻底控制数据和 tokenizer。第二，训练数据的单位是什么？file-level、repo-level、PR-level 和 issue-level 代表不同的软件工作视角。

第三，训练目标是否包含编辑能力？如果只有 next-token prediction，模型可能很擅长续写，却不擅长在已有代码中间修改。FIM、repo-level FIM 和 patch-style data 都是在回答这个问题。第四，上下文长度如何获得？是直接扩大窗口，还是用 long-context fine-tuning、RoPE/YARN 调整和结构化文件边界让模型真正适应长输入？

第五，后训练把模型接到什么接口？SFT、DPO、reward modeling、RL、self-instruct 或 executable feedback 会让模型更像助手，但也会改变底层 completion 行为。一个报告如果只给 benchmark 表，却没有说明这些变量，就很难判断模型到底强在哪里。

Five questions: origin model → data unit → edit objective → context method → alignment interface.

这五个问题也能帮助选择模型。如果目标是本地 IDE 补全，小模型、低延迟、FIM 和稳定 completion 更重要。如果目标是 repo-level 修复，repo context、长上下文、测试反馈理解和 instruction following 更重要。如果目标是 agentic rollout，模型还必须能在多轮 observation 中保持目标，并把失败解释转成下一步动作。

13 几个常见误读

参数量不是路线本身。

参数量越大，通常容量越强，但 coder 模型路线不能只看参数量。Code Llama 展示的是通用模型继续代码训练；StarCoder2 展示的是开放数据和 repo-context；DeepSeek-Coder 展示的是项目级数据构造；DeepSeek-Coder-V2 展示的是 MoE 和长上下文；Qwen2.5-Coder 展示的是 file-to-repo-to-alignment。它们回答的问题不同。把这些差异压缩成“谁参数更大”会丢掉最有价值的信息。

长上下文不是仓库理解。

长上下文只是容量；仓库理解需要结构。一个模型可以支持 128K tokens，但如果输入里没有文件边界、路径、依赖、检索排序和测试信号，它仍然可能把无关文件当成关键证据。repo-level pretraining 的意义是让模型在训练时就见过结构化工作区，而不是只在推理时临时塞入更多 token。

repo-level pretraining 不是 retrieval。

repo-level pretraining 让模型学会仓库结构的统计规律；retrieval 是运行时选择哪些文件给模型。二者互补。没有 repo-level pretraining，模型即使拿到相关文件，也可能不懂路径和跨文件关系；没有 retrieval，模型即使懂仓库结构，也可能根本看不到关键文件。coding agent 需要两者配合。

Instruct 不总是比 Base 更适合所有接口。

Instruct model 更适合回答人类任务，但 base model 可能在低层 completion、批量候选生成或继续训练上更干净。Chat alignment 可能让模型更会解释，也可能让它在某些自动化场景中输出多余文字。选择 base、instruct 或 chat，应该取决于 harness 要模型输出什么：纯代码、diff、命令、解释，还是多轮计划。

benchmark 表是终点，不是解释。

HumanEval、MBPP、LiveCodeBench、Aider 或 SWE-bench 数字可以证明某个设置有用，但不能单独说明为什么有用。要解释结果，必须回到训练路线：数据是否更接近目标任务？FIM 是否改善编辑？长上下文是否让模型看到必要信息？alignment 是否改变了输出格式？没有这些问题，数字只是压缩过的现象。

14 模型如何放进 Agent 系统

同一个 coder model 在不同 agent 系统里可以扮演不同角色。最简单的方式是把它当成主模型：负责读任务、选动作、写 patch、解释结果。这个设置对模型要求最高，尤其需要 instruction following、长上下文、错误解释和稳定格式。更常见的系统会把模型拆成多个用途：一个模型做检索 query，一个模型生成候选 patch，一个模型解释测试失败，另一个模型做最终总结。

系统角色	更需要的模型能力	典型选择
主 agent	指令、计划、长上下文	strong instruct
补全器	低延迟、FIM、局部质量	small coder base
候选生成	多样性、代码生成速度	cheap/base model
失败解释	日志理解、推理、自然语言	reasoning/instruct
代码审查	风格、风险、测试意识	aligned coder

这个拆分说明，foundation coder model 的路线不是为了选出唯一冠军，而是为了给系统设计提供组合空间。一个 32B instruct model 可能适合做主 agent，但本地 IDE 需要的是响应很快的 7B 或更小模型；一个 MoE 大模型可能适合复杂跨文件修复，但候选 patch 的初筛可以用更便宜的模型；一个擅长 FIM 的 base model 可以给编辑器提供局部修改，却不一定适合直接和用户长对话。

从这个角度看，模型报告里的很多细节都有系统含义。FIM 说明模型是否适合编辑器和 patch workflow；repo-level data 说明它是否更容易利用跨文件上下文；instruction tuning 说明它是否能接自然语言任务；MoE active parameters 说明它是否能进入多轮工具循环；长上下文说明它能否承载更大的 observation。把这些因素合在一起，才是模型选型。

也正因为如此，agent 系统不应该只记录“用了哪个模型”。一个可复现的系统说明至少要写清：base 还是 instruct，context length，是否启用 FIM 格式，是否有 repo-specific special tokens，工具输出格式是什么，是否允许多候选，验证失败如何反馈给模型。否则，后续看到 pass rate 时，很难判断是模型路线带来的能力，还是 harness 让模型更容易发挥。

15 局限与开放问题

第一，benchmark 仍然会混入 harness 效应。很多报告给出的 HumanEval、MBPP、LiveCodeBench 或 SWE-bench 数字，不能直接解释为纯 base-model 能力。prompt、采样、工具、验证和候选选择都会改变结果。读这些数字时，应当回到 formal #02 的分解：模型、harness、上下文、验证和预算共同决定观测分数。

第二，长上下文还没有解决上下文选择问题。128K 让模型能看更多，但真实仓库可能远超 128K，而且关键文件通常只占很小一部分。没有检索、repo map、AST、LSP 或测试反馈，长上下文只是增大了输入容量，没有自动产生注意力纪律。

第三，repo-level 数据既保留结构，也保留噪声。真实仓库里有重复代码、坏风格、废弃文件、生成文件、依赖混乱和历史包袱。训练时保留仓库结构是必要的，但必须配合质量筛选、去重和安全处理，否则模型会学习到错误的软件工程习惯。

第四，instruction alignment 可能改变原始 completion 行为。模型越像助手，越会解释、拒绝、总结和遵守格式；但某些低层代码补全能力可能会被后训练分布影响。因此 base、instruct、chat、agent model 不能随意混用，它们适合不同接口。

最后，foundation coder model 还不是可执行训练系统。FIM、repo context 和 instruction tuning 让模型更接近 coding assistant；真正的 agentic coding 还要把模型放进 Docker、shell、编辑器、测试和验证闭环中训练或评估。这是下一层系统问题。

16 最终 takeaway

Coder 基座模型的路线可以压缩成三句话：

- 从 completion 到 agent-ready，中间要经过 edit objective、repo context、long context 和 alignment。
- FIM 与 repo-level data 是把模型训练接口贴近真实软件编辑的关键步骤。
- 强 coder 模型是 coding agent 的底座，但 agent 能力仍然取决于 harness、上下文、工具和验证闭环。

检查一个 coder model 是否适合 agent：看它是否学过编辑，是否理解仓库边界，是否能处理长 observation，是否能稳定遵守任务格式，是否能以可接受成本进入多轮工具循环。

实践读法

先看接口，再看分数。如果一个模型主要按 completion 训练，它适合做低延迟补全；如果系统需要在已有文件中插入 patch，就要优先看 FIM 和编辑格式；如果模型要直接承担 issue 理解、计划、命令输出和失败总结，就要看 instruction alignment 与长对话稳定性。benchmark 分数只有放回接口里才有意义。

先看工作集，再看上下文长度。长上下文不是越长越好，而是要和仓库组织方式配合。小模型加检索可能胜过大模型硬塞全文；repo-level 预训练能帮助模型理解跨文件依赖，但仍需要 harness 给出干净的文件子集、测试输出和错误栈。真正的系统能力来自“模型能看多少”和“系统该给它看什么”的配合。

先看反馈闭环，再看模型规模。能运行测试、能回滚 patch、能比较候选的 agent，可以用较便宜模型承担一部分候选生成；反馈稀缺的场景则更依赖强 instruct model 的稳健推理和保守修改。也就是说，foundation coder model 不是孤立采购项，而是 agent architecture 里的一个可替换部件。

参考资料

- [1] [Code Llama](#). open foundation models for code
- [2] [StarCoder2 / The Stack v2](#). open data and repo context
- [3] [DeepSeek-Coder](#). project-level code pretraining
- [4] [DeepSeek-Coder-V2](#). MoE and 128K context
- [5] [Qwen2.5-Coder](#). file-to-repo-to-alignment route